ROS 2 - Sistema Operativo de Robótica

Proyecto de investigación - Sistemas de Comunicaciones

Pablo Sebastian Gomez · Irina Terebiznik · Matias Peña

Sistemas de Comunicaciones



Universidad Tecnológica Nacional
Facultad Regional Avellaneda
Departamento de Ingeniería Electrónica

Índice

1. Capítulo 1 **3**

1.1. Introducción del trabajo . . . 3
1.2. Objetivo y alcance . . . 3
1.3. Metodología y Fuentes de investigación . . . 4
1.4. Delimitación del estudio . . . 5
1.5. Universo Espacio Temporal . . . 5

2. Capítulo 2 **5**

2.1. Introducción del tema . . . 5
2.2. Descripción del contexto del tema . . . 6
2.3. Mercado y Segmentación del tema . . . 7
2.4. Legislación del tema . . . 8

3. Capítulo 3 **9**

3.1. Fundamentos técnicos . . . 9
Conceptos arquitectónicos: nodos y *topics* . . . 9
Patrones de comunicación: *services* y *actions* . . . 9
Arquitectura de ROS 1: modelo híbrido y proceso maestro . . . 10
Arquitectura de ROS 2: modelo descentralizado basado en DDS . . . 11
Calidad de servicio (QoS) . . . 12
Capa de abstracción de *middleware* (RMW) . . . 13
Soporte de tiempo real y microcontroladores . . . 13
3.2. Ámbitos de aplicación . . . 14
3.3. Estrategias Competitivas . . . 15
3.4. Normativas asociadas . . . 16

4. Capítulo 4 **16**

4.1. Conclusiones . . . 17
4.2. Glosario de términos . . . 17
4.3. Referencias . . . 17

Extracto

El presente informe analiza ROS (*Robot Operating System*) desde la perspectiva de los sistemas de comunicaciones. Se describen sus conceptos arquitectónicos y patrones de comunicación, y se examina la transición desde la arquitectura híbrida de ROS 1, dependiente de un proceso maestro central, hacia el modelo descentralizado de ROS 2, sustentado en el estándar DDS y en el protocolo RTPS. Se analizan, asimismo, los mecanismos de calidad de servicio (QoS), la capa de abstracción de *middleware* (RMW) y sus principales implementaciones, el soporte para sistemas de tiempo real y microcontroladores, y el esquema de distribuciones del proyecto. El estudio se complementa con una prueba de concepto sobre hardware dedicado.

1. Capítulo 1

1.1. Introducción del trabajo

El presente trabajo de investigación se desarrolla en el marco de la asignatura Sistemas de Comunicaciones de la Universidad Tecnológica Nacional. Su propósito es analizar ROS (*Robot Operating System*) desde la perspectiva de los sistemas de comunicaciones, considerando los mecanismos de intercambio de información que emplea y los estándares sobre los que se sustenta.

Los sistemas robóticos actuales constituyen, en esencia, sistemas distribuidos. Sensores, actuadores, unidades de procesamiento y estaciones de supervisión operan como procesos independientes, frecuentemente alojados en distintos equipos, que deben intercambiar datos de manera continua a través de una red. En este contexto, la arquitectura de comunicaciones deja de ser un aspecto secundario y pasa a condicionar directamente la latencia, la confiabilidad y la escalabilidad del sistema en su conjunto.

ROS constituye uno de los *frameworks* de código abierto más difundidos para el desarrollo de software robótico, tanto en el ámbito académico como en el industrial. Su evolución resulta particularmente relevante para el estudio de las comunicaciones: la primera versión (ROS 1) se basa en una arquitectura híbrida que depende de un proceso maestro central, mientras que ROS 2 adopta un esquema descentralizado sustentado en el estándar DDS (*Data Distribution Service*) y en el protocolo RTPS (*Real-Time Publish-Subscribe*), que opera habitualmente sobre UDP/IP. Este trabajo aborda dicha transición, analizando sus fundamentos técnicos, la configuración de la calidad de servicio (QoS) y las implementaciones de *middleware* disponibles.

1.2. Objetivo y alcance

El objetivo general del trabajo es analizar el estado del arte de ROS, con foco en la evolución de su arquitectura de comunicaciones desde ROS 1 hasta ROS 2 y en los estándares y protocolos que la sustentan.

Para ello se plantean los siguientes objetivos específicos:

- Describir los conceptos arquitectónicos de ROS (nodos y *topics*) y sus patrones de comunicación (*services* y *actions*).

- Analizar la arquitectura híbrida de ROS 1 y las limitaciones derivadas de su dependencia del proceso maestro (ROS Master) como punto único de falla.
- Describir el modelo descentralizado de ROS 2 y su pila de protocolos: DDS, RTPS y UDP/IP.
- Analizar las políticas de calidad de servicio (QoS) y su configuración.
- Relevar el soporte de ROS 2 para sistemas operativos de tiempo real (RTOS) y para microcontroladores mediante micro-ROS.
- Describir la capa de abstracción de *middleware* (RMW) y comparar sus principales implementaciones: Eclipse Cyclone DDS, eProsima Fast DDS y RTI Connext DDS.
- Presentar el esquema de distribuciones de ROS y su vinculación con las versiones de Ubuntu.
- Realizar una demostración práctica sobre hardware dedicado (Raspberry Pi) que ilustre el intercambio de mensajes entre nodos de ROS 2 a través de la red.

En cuanto al alcance, el estudio se circunscribe a las capas de *middleware* y de transporte de datos, y se complementa con una prueba de concepto de carácter demostrativo. Quedan fuera de su alcance los algoritmos de percepción, planificación y control, así como el desarrollo de aplicaciones robóticas específicas.

1.3. Metodología y Fuentes de investigación

La investigación es de tipo descriptivo y de enfoque cualitativo. Se basa en el análisis documental de las fuentes técnicas que definen la arquitectura de ROS y de los estándares de comunicaciones sobre los que se apoya. Los resultados del análisis se contrastan mediante una prueba de concepto sobre hardware dedicado.

Las fuentes consultadas se agrupan en las siguientes categorías:

- **Documentación oficial de ROS:** documentación de ROS~2 y propuestas de mejora (REP, *ROS Enhancement Proposals*), en particular la REP~2000, que define las plataformas soportadas por cada distribución.
- **Especificaciones de estándares:** especificaciones de la OMG (*Object Management Group*) para DDS y DDSI-RTPS, y documentos RFC de la IETF para los protocolos UDP (RFC~768) e IPv4 (RFC~791).
- **Documentación de fabricantes:** documentación técnica de las implementaciones Eclipse Cyclone DDS, eProsima Fast DDS y RTI Connext DDS, y del proyecto micro-ROS.
- **Publicaciones académicas:** artículos científicos sobre el diseño, la arquitectura y el desempeño de ROS 2 y DDS.

1.4. Delimitación del estudio

El análisis se delimita según los siguientes criterios:

- **Sectores:** telecomunicaciones, robótica y dispositivos IoT (*Internet of Things*).
- **Objeto de estudio:** el subsistema de comunicaciones de ROS 1 y ROS 2, con exclusión del resto de las herramientas del ecosistema.
- **Capas analizadas:** *middleware* (RMW y DDS), protocolo de interoperabilidad (RTPS) y transporte (UDP/IP).
- **Plataformas:** distribuciones de ROS sobre Ubuntu Linux, sistemas operativos de tiempo real y microcontroladores mediante micro-ROS.

1.5. Universo Espacio Temporal

En el plano temporal, el estudio abarca la evolución de ROS desde el inicio de su desarrollo en 2007 hasta las distribuciones vigentes de ROS 2 en 2026. Se pone especial énfasis en el período 2021–2026, durante el cual ROS 2 supera a ROS 1 en adopción y este último alcanza el fin de su soporte con la distribución Noetic, en mayo de 2025.

En el plano espacial, el análisis tiene alcance global, dado que ROS y los estándares asociados son desarrollados y adoptados a nivel internacional. Se incorporan referencias al contexto argentino cuando resultan pertinentes.

2. Capítulo 2

2.1. Introducción del tema

ROS (*Robot Operating System*) es un *framework* de código abierto para el desarrollo de software robótico. A pesar de su denominación, no constituye un sistema operativo en sentido estricto: se ejecuta sobre un sistema operativo anfitrión, principalmente distribuciones de Linux, y provee una capa de *middleware* junto con un conjunto de bibliotecas y herramientas. Su función central es permitir que los distintos componentes de un robot se desarrollen como procesos independientes y se comuniquen entre sí mediante interfaces estandarizadas.

El proyecto se originó en 2007 en el Laboratorio de Inteligencia Artificial de la Universidad de Stanford, en el marco del proyecto STAIR (*STanford Artificial Intelligence Robot*), y continuó su desarrollo en la empresa Willow Garage. Actualmente está a cargo de la Open Source Robotics Foundation (OSRF) y, desde 2024, su gobernanza técnica es ejercida por la Open Source Robotics Alliance (OSRA), con la participación de una amplia comunidad internacional.

Los principales factores que explican su adopción son:

- **Modularidad:** cada función del robot se implementa como un componente independiente, lo que facilita el desarrollo en paralelo y el mantenimiento.

- **Reutilización:** el ecosistema dispone de un amplio conjunto de paquetes para navegación, percepción, manipulación y controladores de hardware.
- **Abstracción de hardware:** los componentes se comunican a través de interfaces normalizadas, con independencia del fabricante del sensor o actuador.
- **Herramientas de desarrollo:** provee utilidades de visualización, registro de datos, simulación y depuración.
- **Licenciamiento abierto:** el núcleo de ROS 1 se distribuye bajo licencia BSD y el de ROS 2 bajo licencia Apache 2.0. Ambas son licencias permisivas que habilitan su uso académico y comercial.

El diseño de ROS 1 se orientó al caso de uso del robot PR2 de Willow Garage, cuyas características determinantes eran: un único robot, recursos de cómputo de clase estación de trabajo, ausencia de requisitos de tiempo real, conectividad de red de alta calidad y aplicaciones predominantemente de investigación. A medida que su uso se extendió a entornos industriales, sistemas multirrobot y redes inalámbricas de menor confiabilidad, estas hipótesis de diseño se tornaron restrictivas. En respuesta, se desarrolló ROS 2, rediseñado desde su base sobre estándares industriales de comunicaciones.

La diferencia fundamental entre ambas versiones reside en su arquitectura de comunicaciones. ROS 1 emplea un esquema híbrido: un proceso maestro central (ROS Master, iniciado mediante el comando `roscore`) actúa como servicio de registro y resolución de nombres a través de XML-RPC, mientras que el intercambio de datos se realiza directamente entre nodos mediante protocolos propios (TCPROS y UDPROS). En consecuencia, el proceso maestro constituye un punto único de falla: sin él, los nodos no pueden localizarse entre sí ni establecer nuevas comunicaciones. Asimismo, ROS 1 ofrece opciones limitadas para ajustar el comportamiento de la comunicación, sin un modelo de calidad de servicio comparable al de DDS.

ROS 2, en cambio, adopta una arquitectura descentralizada basada en el estándar DDS (*Data Distribution Service*) y su protocolo de interoperabilidad RTPS. Esta elección elimina la dependencia de un proceso central, dado que el descubrimiento de participantes se realiza de forma distribuida, y habilita la configuración de políticas de calidad de servicio (QoS). Su diseño contempla, además, la ejecución en sistemas con requisitos de tiempo real y la integración de microcontroladores mediante micro-ROS. Asimismo, incorpora mecanismos de seguridad basados en la especificación DDS-Security, sin equivalente nativo en ROS 1.

Con la finalización del soporte de ROS 1 en 2025, ROS 2 se ha consolidado como la plataforma vigente para el desarrollo de nuevos sistemas robóticos.

Los aspectos técnicos de cada arquitectura se desarrollan en detalle en el Capítulo 3.

2.2. Descripción del contexto del tema

ROS se emplea como capa de integración en sistemas robóticos de diversa complejidad, desde plataformas educativas y de investigación hasta vehículos autónomos y robots industriales. En es-

tos sistemas, los componentes de software suelen distribuirse entre varios equipos: computadoras embarcadas en el robot, microcontroladores asociados a sensores y actuadores, y estaciones de supervisión remotas. ROS actúa como el *middleware* que permite a estos componentes intercambiar información de forma transparente respecto de su ubicación física en la red.

El software de ROS se publica en **distribuciones**: conjuntos versionados de paquetes que se prueban y se mantienen en conjunto. La primera distribución de ROS 2 se publicó en diciembre de 2017 y, durante sus primeros años, las versiones se lanzaron con una frecuencia semestral. Desde 2020 se publica una nueva distribución por año, en el mes de mayo. Las distribuciones de los años pares son de soporte extendido (LTS, *Long Term Support*) y reciben actualizaciones durante cinco años, mientras que las de los años impares tienen un ciclo de soporte de aproximadamente un año y medio. En paralelo existe Rolling, una distribución de desarrollo continuo en la que se incorporan los cambios antes de su inclusión en una versión estable.

Cada distribución está vinculada a una versión LTS específica de Ubuntu, que constituye su plataforma principal de soporte (*Tier 1*), según lo establecido en la REP 2000. Este vínculo se debe a que los paquetes binarios de ROS se compilan y se prueban contra las bibliotecas del sistema de esa versión de Ubuntu. En consecuencia, la elección de la distribución de ROS determina la versión del sistema operativo a utilizar. Las distribuciones LTS más recientes y su correspondencia son:

- **ROS 1 Noetic Ninjemys** (2020): Ubuntu 20.04. Última distribución de ROS 1, con fin de soporte el 31 de mayo de 2025.
- **ROS 2 Humble Hawksbill** (2022): Ubuntu 22.04, con soporte hasta mayo de 2027.
- **ROS 2 Jazzy Jalisco** (2024): Ubuntu 24.04, con soporte hasta mayo de 2029.
- **ROS 2 Lyrical Luth** (2026): Ubuntu 26.04, con soporte hasta mayo de 2031.

Además de Ubuntu, las distribuciones de ROS 2 ofrecen distintos niveles de soporte para otras plataformas, como Windows y Red Hat Enterprise Linux.

2.3. Mercado y Segmentación del tema

Al tratarse de un proyecto de código abierto y libre distribución, ROS no constituye un producto comercial en sí mismo. Su mercado se define, en cambio, por el ecosistema de organizaciones que lo adoptan como base para sus desarrollos y por las empresas que ofrecen productos y servicios compatibles.

A nivel mundial, la adopción de ROS puede segmentarse según los siguientes criterios:

- **Por sector de aplicación:** robótica industrial y colaborativa, logística y almacenes (robots móviles autónomos), vehículos autónomos, agricultura de precisión, robótica de servicio y salud, y aplicaciones espaciales.
- **Por tipo de usuario:** universidades y centros de investigación, empresas emergentes que desarrollan productos robóticos, grandes empresas industriales y fabricantes de hardware que proveen controladores compatibles con ROS para sus sensores, actuadores y robots.

- **Por versión:** si bien ROS 1 mantuvo una amplia base instalada durante más de una década, la adopción se orientó progresivamente hacia ROS 2: hacia septiembre de 2024, ROS 2 ya representaba casi el 80 % de las descargas totales de ROS. La finalización del soporte de ROS 1 en 2025 consolidó esta tendencia.

La adopción industrial se apoya en iniciativas como el consorcio ROS-Industrial, orientado a extender ROS para la automatización de la manufactura, y Autoware, una plataforma de código abierto para la conducción autónoma construida sobre ROS 2. Asimismo, la constitución de la OSRA en 2024 incorporó un modelo de membresía que suma la participación y el financiamiento de empresas del sector, como NVIDIA, Intrinsic y Qualcomm.

En Argentina, el uso de ROS se concentra principalmente en el ámbito académico, en universidades y grupos de investigación en robótica, automatización y control. En el ámbito productivo, su aplicación presenta un desarrollo incipiente, con mayor potencial en sectores como la agroindustria, la automatización de procesos industriales y la logística.

2.4. Legislación del tema

No se ha identificado legislación que regule específicamente a ROS ni a los *middleware* de comunicaciones robóticas. Sin embargo, los sistemas construidos sobre ROS quedan alcanzados por la normativa general aplicable a máquinas, productos con componentes digitales y tratamiento de datos. Las normas técnicas asociadas se desarrollan en la Sección 3.4.

En el contexto internacional, la Unión Europea constituye la principal referencia regulatoria:

- **Reglamento (UE) 2023/1230 relativo a las máquinas:** reemplaza a la Directiva 2006/42/CE y será de aplicación a partir del 20 de enero de 2027. Incorpora requisitos sobre el software que cumple funciones de seguridad y sobre la protección frente a ciberataques que puedan comprometer la operación segura de la máquina.

- **Reglamento (UE) 2024/2847 de Ciberresiliencia:** establece requisitos de ciberseguridad para productos con elementos digitales. Entró en vigor en diciembre de 2024 y será plenamente aplicable a partir del 11 de diciembre de 2027. Su alcance comprende a los robots conectados en red. Prevé un régimen diferenciado para el software de código abierto desarrollado sin fines comerciales.

- **Reglamento (UE) 2024/1689 de Inteligencia Artificial:** resulta aplicable a los sistemas robóticos que incorporan componentes de inteligencia artificial, según su nivel de riesgo.

En Argentina no se ha identificado legislación sancionada que regule específicamente la robótica. Los sistemas robóticos quedan alcanzados por normas de carácter general:

- **Ley 19.587 de Higiene y Seguridad en el Trabajo** (1972) y su decreto reglamentario 351/79, aplicables a la incorporación de robots en entornos laborales.

- **Ley 25.326 de Protección de los Datos Personales** (2000), aplicable cuando los robots capturan y transmiten información que permite identificar a personas, como imágenes o

audio.

- **Ley 11.723 de Propiedad Intelectual**, que comprende al software tras la modificación introducida por la Ley 25.036 (1998), y que da el marco legal a las licencias de código abierto bajo las que se distribuye ROS.

3. Capítulo 3

3.1. Fundamentos técnicos

Conceptos arquitectónicos: nodos y *topics*

Un sistema basado en ROS se organiza como un grafo de procesos que se comunican entre sí, denominado grafo de ROS. La unidad básica de este grafo es el **nodo**: un participante que realiza una función lógica específica, como leer un sensor, estimar la posición del robot o controlar un motor. Cada nodo utiliza una biblioteca cliente para comunicarse con el resto del sistema: `roscpp` y `rospy` en ROS 1, y `rclcpp` y `rclpy` en ROS 2, para los lenguajes C++ y Python respectivamente. Los nodos pueden comunicarse con otros nodos alojados en el mismo proceso, en otro proceso o en otro equipo de la red. Las conexiones entre ellos se establecen mediante un mecanismo de descubrimiento, cuya implementación difiere entre ROS 1 y ROS 2, según se detalla más adelante.

El mecanismo de comunicación fundamental es el ***topic***: un canal con nombre sobre el que opera un modelo de publicación/suscripción. Los nodos productores de datos (*publishers*) publican mensajes en un *topic*, y todos los nodos suscritos a él (*subscribers*) los reciben. Un mismo *topic* admite cero o más publicadores y cero o más suscriptores, lo que configura un esquema de comunicación de muchos a muchos. Este modelo presenta dos propiedades relevantes:

- **Anonimato:** el suscriptor no necesita conocer qué nodo originó el dato. Esto permite reemplazar, agregar o quitar nodos sin modificar el resto del sistema.
- **Tipado fuerte:** cada *topic* transporta un único tipo de mensaje, definido en un archivo `.msg`. A partir de esta definición, las herramientas de ROS generan automáticamente el código para los distintos lenguajes soportados.

Los *topics* resultan adecuados para flujos de datos continuos, como las lecturas de sensores o el estado del robot, donde no se espera una respuesta por parte del receptor.

Patrones de comunicación: *services* y *actions*

Además de los *topics*, ROS ofrece dos patrones de comunicación orientados a la interacción entre un cliente y un servidor.

Un ***service*** implementa una llamada a procedimiento remoto (RPC, *Remote Procedure Call*) bajo un esquema de solicitud y respuesta. Un nodo cliente envía una solicitud, el nodo servidor realiza el procesamiento correspondiente y devuelve un resultado. Su interfaz se define en un archivo `.srv`, compuesto por un mensaje de solicitud y uno de respuesta. Cada nombre de *service* debe

tener un único servidor, aunque puede ser utilizado por cualquier cantidad de clientes. Dado que el cliente generalmente queda a la espera del resultado, los *services* están pensados para operaciones de ejecución breve, como consultar un estado o modificar una configuración.

Una ***action*** es una llamada a procedimiento remoto de larga duración, que incorpora la posibilidad de recibir retroalimentación durante la ejecución y de cancelar el objetivo antes de su finalización. Un ejemplo típico es la orden de desplazar el robot hasta una posición determinada: la tarea puede demorar segundos o minutos, durante los cuales el servidor informa el progreso y el cliente puede cancelarla. Su interfaz se define en un archivo `.action`, compuesto por tres mensajes: objetivo (*goal*), resultado (*result*) y retroalimentación (*feedback*).

En ROS 2, las *actions* no constituyen un mecanismo de transporte independiente, sino que se construyen sobre los dos patrones anteriores: cada *action* se compone internamente de tres *services* (envío del objetivo, cancelación y obtención del resultado) y dos *topics* (retroalimentación y estado). En ROS 1, en cambio, las *actions* no formaban parte del núcleo y se implementaban mediante una biblioteca independiente denominada `actionlib`.

Arquitectura de ROS 1: modelo híbrido y proceso maestro

En ROS 1, el grafo se organiza en torno a un proceso maestro (ROS Master), que se inicia mediante el comando `roscore` junto con el servidor de parámetros (*Parameter Server*) y el nodo de registro `rosout`. El proceso maestro provee servicios de registro y resolución de nombres: cada nodo registra ante él los *topics* que publica, aquellos a los que se suscribe y los *services* que ofrece. La comunicación con el proceso maestro se realiza mediante llamadas XML-RPC, y su ubicación se indica a cada nodo a través de la variable de entorno `ROS_MASTER_URI`, cuyo puerto por defecto es el 11311.

El establecimiento de una comunicación sobre un *topic* sigue la secuencia que se describe a continuación:

1. El nodo publicador registra el *topic* ante el proceso maestro.
2. El nodo suscriptor registra su interés en el *topic* y recibe del proceso maestro la lista de publicadores activos.
3. El suscriptor contacta directamente a cada publicador, también mediante XML-RPC, para negociar el protocolo de transporte.
4. El intercambio de mensajes se realiza directamente entre ambos nodos, sin intervención del proceso maestro.

Para el intercambio de datos, ROS 1 define dos protocolos propios. TCPROS, basado en TCP, es el transporte por defecto y el único que toda biblioteca cliente debe implementar. UDPROS, basado en UDP, ofrece menor latencia a costa de admitir pérdidas, por lo que resulta adecuado para aplicaciones como la teleoperación; sin embargo, solo está disponible en la biblioteca `roscpp`. En ambos casos, cada par publicador-suscriptor establece una conexión independiente, de modo que un mismo mensaje se transmite tantas veces como suscriptores remotos existan.

La arquitectura resultante es híbrida: centralizada para el descubrimiento y distribuida para el transporte de datos. Esta característica determina su principal limitación: el proceso maestro constituye un punto único de falla, dado que sin él los nodos no pueden localizarse entre sí ni establecer nuevas comunicaciones. A ello se suman la ausencia de un mecanismo de descubrimiento automático en la red y la falta de un modelo de calidad de servicio que permita adaptar la comunicación a enlaces con pérdidas.

Arquitectura de ROS 2: modelo descentralizado basado en DDS

ROS 2 elimina el proceso maestro y delega el descubrimiento, la serialización y el transporte de datos en un *middleware* que implementa el estándar DDS y su protocolo de interoperabilidad RTPS. El descubrimiento se realiza de forma distribuida: cada participante anuncia su presencia en la red y detecta a los demás sin intermediarios, lo que suprime el punto único de falla de ROS 1.

DDS (*Data Distribution Service*). Es un estándar de la OMG, cuya versión vigente es la 1.4 (2015), que define un modelo de publicación/suscripción centrado en los datos (DCPS, *Data-Centric Publish-Subscribe*). En este modelo, las aplicaciones comparten un espacio global de datos en el que el *middleware* conoce el tipo y las políticas de calidad de servicio de cada flujo de información, y no solo su contenido binario. Sus entidades principales son el dominio, el participante (*DomainParticipant*), el *topic*, y los escritores (*DataWriter*) y lectores (*DataReader*) de datos. ROS 2 asocia sus conceptos a estas entidades: cada proceso crea un participante DDS, los publicadores y suscriptores de ROS 2 se implementan como *DataWriters* y *DataReaders*, y la variable de entorno `ROS_DOMAIN_ID` (por defecto, 0) determina el dominio DDS. Los nodos ubicados en dominios distintos no pueden comunicarse entre sí, lo que permite segmentar lógicamente una misma red física.

RTPS (*Real-Time Publish-Subscribe*). DDS especifica el modelo y la interfaz de programación, pero no el formato de los mensajes en la red. Esta función corresponde al protocolo DDSI-RTPS, también estandarizado por la OMG (versión vigente 2.5, de 2022), que define la representación de los datos en la red y garantiza la interoperabilidad entre implementaciones de distintos fabricantes. RTPS especifica, entre otros aspectos:

- **Descubrimiento en dos fases:** el protocolo SPDP (*Simple Participant Discovery Protocol*) permite que los participantes anuncien su presencia en el dominio, y el protocolo SEDP (*Simple Endpoint Discovery Protocol*) permite intercambiar la información de sus *DataWriters* y *DataReaders*.
- **Confiabilidad sobre transportes no confiables:** mediante los submensajes HEARTBEAT, con el que el escritor informa las muestras disponibles, y ACKNACK, con el que el lector confirma las muestras recibidas y solicita la retransmisión de las faltantes.

UDP (*User Datagram Protocol*). RTPS opera habitualmente sobre UDP, definido en el RFC 768 (1980). Se trata de un protocolo de transporte no orientado a conexión que, según su especificación, no garantiza la entrega ni la protección contra duplicados. Su elección responde a dos motivos: admite el envío *multicast*, necesario para el descubrimiento automático, y su bajo costo permite que RTPS implemente la confiabilidad solo cuando la política de calidad de servicio lo requiere. Por este motivo, incluso la comunicación confiable de ROS 2 se implementa sobre UDP. Las implementaciones pueden emplear, además, mecanismos alternativos como la memoria compartida para la comunicación entre procesos de un mismo equipo.

IP (*Internet Protocol*). En la capa de red, IP, definido para su versión 4 en el RFC 791 (1981), se limita a entregar datagramas desde un origen hasta un destino a través de redes interconectadas. El descubrimiento de RTPS utiliza el grupo *multicast* 239.255.0.1, y los puertos UDP se derivan del identificador de dominio según la expresión $7400 + 250 \cdot d$ más un desplazamiento que depende del tipo de tráfico y del participante, donde d es el identificador de dominio.

En síntesis, la pila de comunicaciones de ROS 2 se organiza en cuatro niveles: DDS define el modelo de datos y la interfaz de programación; RTPS, el protocolo de interoperabilidad, el descubrimiento y la confiabilidad; UDP, el transporte de los datagramas; e IP, su encaminamiento a través de la red.

Calidad de servicio (QoS)

ROS 2 permite configurar el comportamiento de cada comunicación mediante un conjunto de políticas de calidad de servicio heredadas de DDS. La combinación de políticas aplicada a un publicador, un suscriptor, un servidor o un cliente de *service* constituye un perfil de QoS. Las políticas disponibles son:

- ***History*:** *keep last* conserva únicamente las últimas N muestras, mientras que *keep all* conserva todas las muestras dentro de los límites de recursos del *middleware*.
- ***Depth*:** tamaño de la cola, aplicable solo con *keep last*.
- ***Reliability*:** *best effort* intenta entregar las muestras, pero admite pérdidas; *reliable* garantiza la entrega mediante retransmisiones.
- ***Durability*:** con *transient local*, el publicador conserva las muestras para entregarlas a suscriptores que se incorporen tardíamente; con *volatile*, no se conservan.
- ***Deadline*:** intervalo máximo esperado entre mensajes consecutivos.
- ***Lifespan*:** tiempo máximo entre la publicación y la recepción de un mensaje antes de que se lo considere expirado y se lo descarte.
- ***Liveliness* y *Lease Duration*:** mecanismo y plazo con los que un publicador debe indicar que continúa activo antes de que el sistema considere que perdió vigencia.

Para simplificar su uso, ROS 2 define perfiles predefinidos. El perfil por defecto emplea *keep last* con profundidad 10, *reliable* y *volatile*, con el fin de reproducir el comportamiento de ROS 1. El perfil para datos de sensores, en cambio, emplea *best effort* y una cola de menor tamaño, dado que en estas aplicaciones resulta prioritario recibir la muestra más reciente antes que garantizar la recepción de todas. Existen, además, perfiles específicos para *services* y parámetros.

La conexión entre un publicador y un suscriptor solo se establece si sus perfiles son compatibles, según un modelo de solicitud y oferta (*Request vs. Offered*): el suscriptor solicita una calidad mínima y el publicador ofrece una calidad máxima, y la comunicación se establece únicamente si ninguna política solicitada es más exigente que la ofrecida. Por ejemplo, un publicador *best effort* no puede comunicarse con un suscriptor *reliable*, mientras que la combinación inversa sí es válida.

Respecto de ROS 1, las políticas *History* y *Depth* equivalen al tamaño de cola; *Reliability* equivale a la elección entre UDPROS y TCPROS; y *transient local* reproduce el comportamiento de los publicadores con retención (*latching*). Las políticas restantes no tienen equivalente en ROS 1.

Capa de abstracción de *middleware* (RMW)

Para no depender de un único fabricante, ROS 2 interpone una capa de abstracción entre su código y la implementación de DDS. La arquitectura se organiza en los siguientes niveles:

1. **Bibliotecas cliente** (`rclcpp`, `rclpy`): proveen la interfaz de programación que utiliza el desarrollador.
2. **`rcl`** (*ROS Client Library*): biblioteca en C que implementa la lógica común a todas las bibliotecas cliente.
3. **`rmw`** (*ROS Middleware Interface*): interfaz que define la funcionalidad mínima que debe proveer el *middleware*: descubrimiento, publicación y suscripción, solicitud y respuesta, y serialización.
4. **Implementación de `rmw`**: paquete específico de cada fabricante (por ejemplo, `rmw_fastrtps_cpp`) que traduce la interfaz `rmw` a la API de su implementación de DDS o RTPS.

Esta arquitectura aísla al código de usuario del *middleware* subyacente, de modo que el cambio de fabricante no requiere modificar la aplicación. La implementación se selecciona en tiempo de ejecución mediante la variable de entorno `RMW_IMPLEMENTATION`; si no se especifica, se utiliza Fast DDS. Las distintas implementaciones pueden comunicarse entre sí al compartir el protocolo RTPS, aunque la documentación oficial advierte que la interoperabilidad entre fabricantes no está garantizada en todos los casos.

Soporte de tiempo real y microcontroladores

ROS 1 no fue diseñado para aplicaciones de tiempo real: sus hipótesis de diseño asumían que tales requisitos se resolverían mediante soluciones específicas, externas al sistema. ROS 2 incorpora este requisito desde su concepción, aunque el determinismo temporal no depende únicamente

de ROS, sino de toda la pila de ejecución: el sistema operativo, el *middleware*, la planificación de la ejecución de las tareas y la gestión de memoria de la aplicación.

En plataformas Linux, el comportamiento de tiempo real se obtiene mediante el conjunto de modificaciones PREEMPT_RT, que fue incorporado a la rama principal del núcleo de Linux en la versión 6.12, tras cerca de dos décadas de desarrollo independiente. El grupo de trabajo de tiempo real de ROS 2 (*Real-Time Working Group*) recomienda su uso en aplicaciones con estos requisitos y desarrolla herramientas de análisis y medición de desempeño.

Para dispositivos con recursos limitados, el proyecto micro-ROS extiende ROS 2 a microcontroladores que ejecutan sistemas operativos de tiempo real (RTOS), como FreeRTOS, Zephyr y NuttX. Su arquitectura se basa en el modelo cliente-agente del *middleware* Micro XRCE-DDS, que implementa el estándar DDS-XRCE (*DDS for eXtremely Resource Constrained Environments*) de la OMG:

- **Cliente:** biblioteca escrita en C99 que se ejecuta en el microcontrolador y opera sin memoria dinámica.
- **Agente:** aplicación que se ejecuta en un equipo con mayores recursos y crea, en representación del cliente, las entidades DDS correspondientes.

De este modo, el microcontrolador se presenta ante el resto del sistema como un participante más del grafo de ROS 2. La comunicación entre cliente y agente puede realizarse a través de enlaces serie o de UDP, entre otros transportes.

3.2. Ámbitos de aplicación

ROS se aplica en un amplio espectro de dominios, en los que su ecosistema provee componentes especializados. Según el informe de métricas del proyecto correspondiente a 2022, más de 740 empresas comerciales utilizaban ROS. Entre los principales ámbitos se destacan:

- **Investigación y educación:** ámbito de origen del proyecto, en el que ROS constituye una herramienta habitual para la enseñanza y el desarrollo de prototipos robóticos.
- **Robótica industrial:** el consorcio ROS-Industrial, impulsado en 2012 por el Southwest Research Institute (SwRI), extiende ROS a las necesidades de la manufactura. Cuenta con consorcios regionales en América, Europa (coordinado por Fraunhofer IPA) y Asia.
- **Robótica móvil y logística:** Nav2, la evolución para ROS 2 de la pila de navegación de ROS, provee funciones de localización, planificación de trayectorias y evasión de obstáculos para robots móviles.
- **Manipulación:** MoveIt 2 constituye el *framework* de planificación de movimientos para brazos robóticos en ROS 2.
- **Vehículos autónomos:** Autoware es una pila de software de código abierto para conducción autónoma, construida sobre ROS 2 y distribuida bajo licencia Apache 2.0, que abarca

percepción, localización, predicción, planificación y control del vehículo.

- **Aplicaciones espaciales:** el robot Robonaut 2 de la NASA constituyó el primer uso confirmado de ROS en el espacio, a bordo de la Estación Espacial Internacional hacia 2014. Actualmente, el proyecto Space ROS, impulsado por la NASA, Blue Origin y Open Robotics, adapta ROS 2 a los requisitos de verificación y validación del software aeroespacial.

3.3. Estrategias Competitivas

ROS no compite como producto comercial, sino como plataforma abierta. Su estrategia se sustenta en tres elementos: el licenciamiento permisivo, que permite su uso comercial sin restricciones significativas; la adopción de estándares abiertos de comunicaciones, que evita la dependencia de tecnologías propietarias; y el ecosistema de paquetes y herramientas, que reduce el costo de desarrollo de nuevos sistemas.

La competencia más relevante desde la perspectiva de las comunicaciones se produce en la capa de *middleware*. La interfaz RMW permite que distintas implementaciones compitan por ser adoptadas sin afectar a las aplicaciones. En la distribución Lyrical Luth, cuatro implementaciones cuentan con soporte de primer nivel (*Tier 1*), con modelos de negocio diferenciados:

- **eProsima Fast DDS:** implementación de código abierto bajo licencia Apache 2.0. Es el *middleware* por defecto de todas las distribuciones LTS de ROS 2 y se distribuye con sus paquetes binarios. eProsima desarrolla, además, Micro XRCE-DDS, el *middleware* de micro-ROS.

- **Eclipse Cyclone DDS:** implementación de código abierto bajo licencia EPL 2.0 (*Eclipse Public License*), gobernada por la Eclipse Foundation. Tiene su origen en la contribución de la empresa ADLINK a partir de la tecnología OpenSplice DDS.

- **RTI Connext DDS:** implementación comercial de Real-Time Innovations (RTI), disponible sin costo para uso no comercial y de investigación. RTI ofrece variantes orientadas a microcontroladores y a aplicaciones que requieren certificaciones de seguridad funcional, si bien ROS 2 solo soporta su versión estándar.

- **Eclipse Zenoh:** alternativa que no se basa en DDS. Utiliza un enrutador para el descubrimiento, deshabilita el *multicast* por defecto y emplea TCP como transporte, lo que resulta ventajoso en redes donde el *multicast* es poco confiable, como las redes inalámbricas. Se distribuye con los paquetes binarios de ROS 2 desde la distribución Kilted Kaiju.

Existen, asimismo, implementaciones con soporte comunitario, como GurumDDS, de GurumNetworks. Esta pluralidad de proveedores constituye una ventaja estratégica de ROS 2: evita la dependencia de un único fabricante y permite seleccionar el *middleware* según criterios técnicos, como la latencia o el consumo de recursos, o logísticos, como el licenciamiento y el soporte comercial.

3.4. Normativas asociadas

Las normas técnicas vinculadas a ROS se concentran en los estándares de comunicaciones sobre los que se apoya su *middleware*. Se agrupan según el organismo emisor:

- **OMG (*Object Management Group*):**
  - DDS 1.4 (2015): modelo de publicación/suscripción centrado en los datos e interfaz de programación.
  - DDSI-RTPS 2.5 (2022): protocolo de interoperabilidad en la red.
  - DDS-Security 1.2 (2026): autenticación, control de acceso y cifrado, base de SROS 2.
  - DDS-XRCE 1.0 (2020): acceso a DDS desde dispositivos con recursos extremadamente limitados, base de micro-ROS.
  - DDS-XTypes: sistema de tipos extensible para la definición y evolución de los tipos de datos.
- **IETF (*Internet Engineering Task Force*):** RFC 768 (UDP, 1980) y RFC 791 (IPv4, 1981), que definen el transporte y la capa de red sobre los que opera RTPS.
- **IEEE (*Institute of Electrical and Electronics Engineers*):** las normas IEEE 802.3 (Ethernet) e IEEE 802.11 (redes inalámbricas de área local) definen los enlaces sobre los que se despliegan habitualmente los sistemas basados en ROS. No se identificó una norma IEEE específica para ROS o DDS.
- **ITU (*International Telecommunication Union*):** no se identificaron recomendaciones de la ITU aplicables específicamente a ROS o a DDS.
- **ISO (*International Organization for Standardization*):** si bien no regulan el *middleware*, alcanzan a los sistemas robóticos construidos sobre ROS:
  - ISO 10218-1:2025 e ISO 10218-2:2025: requisitos de seguridad para robots industriales, dirigidos a fabricantes e integradores, respectivamente. Reemplazan a las ediciones de 2011.
  - ISO/TS 15066:2016: requisitos de seguridad para aplicaciones de robots colaborativos, complementaria de la ISO 10218.
  - ISO 13482:2014: requisitos de seguridad para robots de cuidado personal.

En el ámbito interno del proyecto, las REP (*ROS Enhancement Proposals*) cumplen una función normativa propia: documentan las convenciones, los procesos y las plataformas soportadas, como la REP 2000 citada en la Sección 2.2.

4. Capítulo 4

4.1. Conclusiones

Al finalizar el ...

4.2. Glosario de términos

Terminología utilizada en el informe:

- **ROS 2:** *Robot Operating System 2*. Segunda generación de ROS, *framework* de código abierto para el desarrollo de software robótico.
- **Nodo:** Participante del grafo de ROS que realiza una función lógica específica y se comunica con otros nodos.

4.3. Referencias

[1] A. S. Sedra y K. C. Smith, *Microelectronic Circuits*, 7ma ed. Oxford University Press, 2015.

[2] Texas Instruments, *LM317 3-Terminal Adjustable Regulator*, Datasheet SLVS044X, 2020. [Disponible en carpeta `references/`].